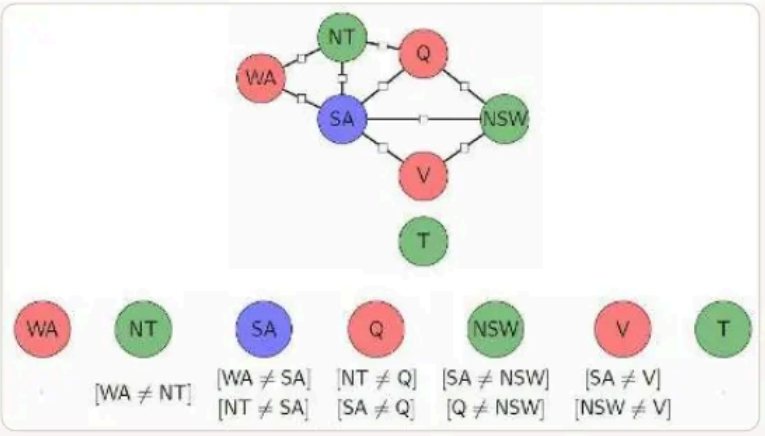


Partial assignment weights & dependent factors (p.5)

Backtracking Search p.7-13

Q. MCV(most constrained var) 휴리스틱?

Q. LCV(least constrained value)?

Q. exact·복잡도 특성은?

Backtracking 기본 아이디어 — DFS로 변수에 값을 하나씩 배정. weight=0 가지치기: partial weight가 0이면 그 서브트리 전체 포기. 완전 배정에 도달하면 best 갱신.

- 시간복잡도: $O(|Domain|^n)$ — 모든 조합 탐색

Backtrack(a, w_best, Domains)

```
if a is complete assignment:
  update best(w_best); return
Choose unassigned VARIABLE X_i     ← MCV heuristic
Order VALUES Domain(X_i)           ← LCV heuristic
for each value v in that order:
  w ← ∏_{f∈dep(a,X_i)} f(a ∪ {X_i:v})
  if w = 0: continue               ← prune
  Domains' ← Domains via LOOKAHEAD ← forward checking / AC-3
  if any Domains'[X] = ∅: continue ← domain wipeout
  Backtrack(a ∪ {X_i:v}, w_best, Domains')
```

w=0 가지치기가 핵심. Lookahead로 domain wipeout 조기 감지.

동적 순서 결정 휴리스틱 — 어떤 변수·값을 먼저 배정하느냐로 탐색 공간을 크게 줄일 수 있다.

MCV=fail fast
LCV=solution 유리

- MCV (Most Constrained Variable) — domain 크기 최소 변수 먼저. 어차피 모든 변수를 배정해야 하므로 *fail early*: 문제 있으면 빨리 감지
- LCV (Least Constrained Value) — 이웃 변수에 남은 일관값 수가 최대인 값 먼저. 해에 도달할 가능성 최대화

✎ 핵심 MCV는 constraint(0/1 factor)가 있을 때 유효. LCV는 모든 factor가 constraint(weight∈{0,1})일 때만 유효. Forward checking이 없으면 domain 정보가 갱신되지 않아 둘 다 무력화됨.

Lookahead: Forward Checking & AC-3 p.8-21

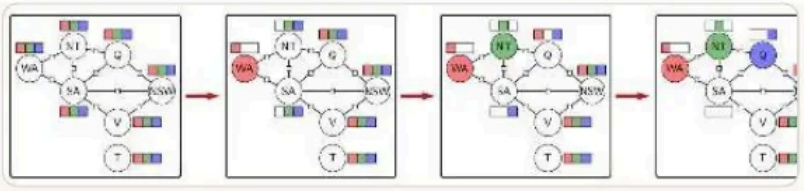
Q. forward checking 동작은?

Q. AC-3 arc consistency 절차?

Q. arc-consistent여도 못 잡는 것은?

Forward Checking (1-step lookahead) — $X_i = v$ 배정 직후, X_i의 모든 이웃 X_j에 대해 비일관 값(x_j s.t. $f(X_i=v, X_j=x_j)=0$)을 Domain(X_j)에서 즉시 제거.

- Domain(X_j)가 비면(empty) → *즉시 return*: 이 브랜치 불가
- 한 단계 이웃만 검사 → 빠르지만 전파 범위 제한적



Forward checking 단계별 domain 갱신 (p.8)

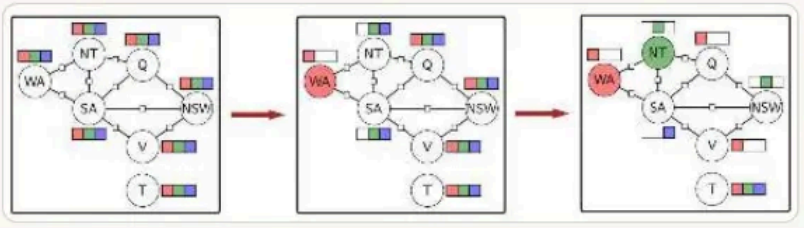
Arc Consistency 정의 — 변수 X_i가 X_j에 대해 **arc-consistent** ↔ $\forall x_i \in \text{Domain}(X_i)$, $\exists x_j \in \text{Domain}(X_j)$: 모든 X_i·X_j 공유 factor에서 $f(X_i=x_i, X_j=x_j)\neq 0$

- 예: $X_i \in \{1,2,3,4,5\}$, $X_j \in \{1,2\}$, factor=$[X_i+X_j=4]$ → X_i를 {2,3}으로 축소

AC-3

```
Queue Q ← {(X_i, X_j) : 모든 arc}
while Q is non-empty:
  Remove any (X_i, X_j) from Q
  EnforceArcConsistency(X_i, X_j)   ← X_i를 X_j 기준으로 축소
  if Domain(X_i) changed:
    for each neighbor X_k of X_i (k≠j):
      add (X_k, X_i) to Q           ← 변화 전파
```

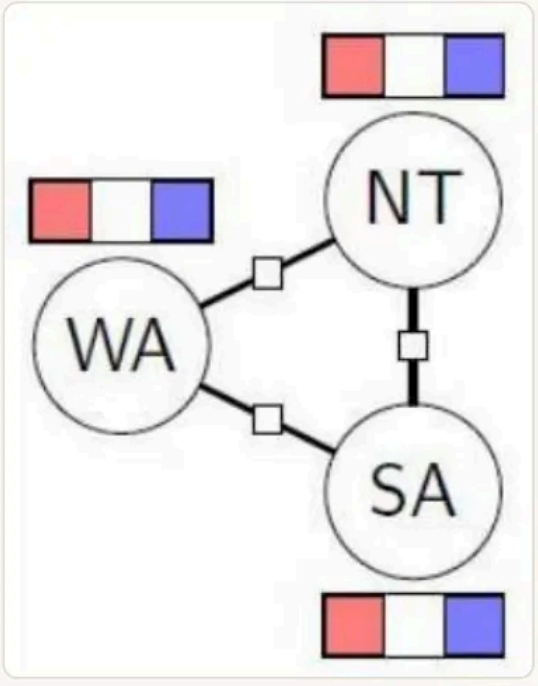
Forward checking = AC-3를 배정된 변수의 이웃에만 1회 실행. AC-3 = 전파 반복.



AC-3 domain 전파 예제: 3단계 (p.18)

AC-3 한계 — **지역적 일관성만 보장**: 쌍(pair)마다 arc-consistent해도 전역(global) 해가 없을 수 있음.

- 예: WA·NT·SA가 모두 arc-consistent지만 3-coloring 불가
- 이유: 삼각형 같은 구조는 두 변수씩 볼 때 문제가 안 보임



AC-3로 검출 불가한 불일치 예 (p.20)

⚠ **함정** AC-3 후 domain이 모두 비어있지 않아도 **해가 없을 수 있다**. Arc consistency ≠ global consistency.

	Forward Checking	AC-3
범위	배정 변수의 이웃 1단계	모든 arc 반복 전파
시점	변수 배정마다 1회	변수 배정마다 반복
효과	즉각 domain 정리	더 넓은 전파, 더 강한 pruning
한계	이웃의 이웃 불검사	pairwise만, global 불일치 미감지

FC vs AC-3 비교

Approximate Search: Beam Search p.23-29

Q. beam width K의 역할은?

Q. 왜 approximate인가? 복잡도?

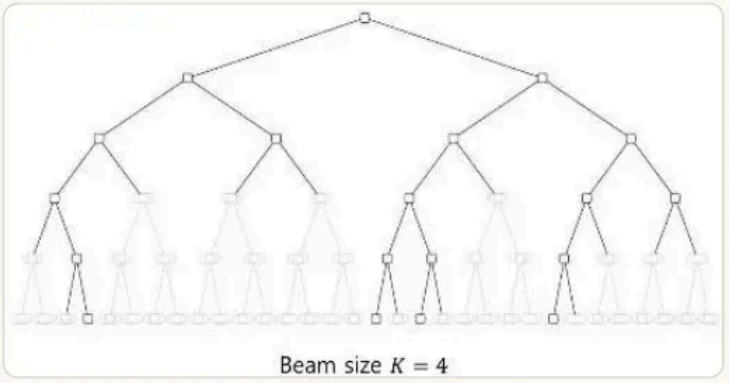
Backtracking vs Beam Search 대비 — Backtracking = *DFS* 정확 탐색. Beam = *pruned BFS* 근사 탐색.

- **Greedy search** (K=1): 각 단계마다 weight 최고인 값 1개만 유지. O(n·|D|)
- **BFS tree search** (K=∞): 모든 partial assignment 유지. O(|D|^n)
- **Beam search** (K 고정): 상위 K개 partial assignment만 유지 — **효율·정확 균형**

Beam Search (beam size K)

```
Initialize C ← [{}]                    ← 빈 partial assignment 1개
for i = 1, …, n:
  Extend:
    C' ← { a ∪ {X_i : v} : a∈C, v∈Domain(X_i) }
  Prune:
    C ← top-K elements of C' by weight   ← sort O(|C'| log |C'|)
return argmax_{a∈C} Weight(a)
```

K=1 → greedy. K=∞ → BFS. 시간: O(n·|D|·K·log(|D|·K)).



Beam size K=4: 각 레벨에서 상위 K만 생존 (p.27)

Beam Search 복잡도 & 특성

K 조정으로 효율·정확 트레이드오프

- 시간: **O(n·|D|·K·log(K·|D|))** — 정렬 비용 포함
- K=1 greedy = O(n) (branching factor |D|로 선형)
- K=∞ BFS = O(|D|^n) (지수)
- **최적해 미보장** — K가 클수록 정확, 작을수록 빠름
- Backtracking:DFS = Beam:BFS (구조적 유사)

★ **필수암기** Backtracking=exact·지수 / Beam=approximate·선형. K=1(greedy) → O(n·|D|). K=∞(BFS) → 지수. Beam: DFS 대응BFS, 순서는 weight 내림차순.

Approximate Search: Local Search (ICM) p.31-36

Q. ICM은 무엇을 반복 수정?

Q. 수렴 보장·전역최적 보장 여부?

Local Search 아이디어 — Backtracking·Beam은 *partial assignment를 확장*. Local search는 **완전 배정을 수정**: 한 번에 변수 하나씩 re-assign.

- 초기 완전 배정(랜덤)에서 시작 → 반복 개선
- Locality 활용: X_i 재배정 시 *X_i에 의존하는 factor만* 다시 계산

ICM (Iterated Conditional Modes) — 완전 배정 유지하며 변수를 순환 방문, 각 변수를 **조건부 최대화**.

단조증가 → 수렴
전역최적 보장 X

- 한 단계: $X_i = \text{argmax}_v \prod_{f: X_i \in \text{scope}(f)} f(a[-X_i] \cup \{X_i:v\})$
- **수렴 보장**: 각 업데이트 후 Weight(a) 단조 증가 또는 유지 → 유한 반복 후 수렴
- **지역 최적(local optimum)**에 수렴 가능 — 전역 최적 미보장

ICM (Iterated Conditional Modes)

```
Initialize a ← random complete assignment
repeat until convergence:
  for i = 1, …, n:
    for each value v in Domain(X_i):
      compute Weight(a ∪ {X_i : v})
    a[X_i] ← argmax_v Weight(a ∪ {X_i : v})   ← conditional mode
return a
```

Weight(a) 단조 비감소 → 유한 수렴. 단, local optimum 가능.

⚠ **함정** ICM은 **수렴은 보장**하지만 **최적해는 미보장**. Local optimum 탈출: 다중 랜덤 초기화(restarts) 또는 Gibbs sampling(확률적 버전).

알고리즘 총 비교 p.23-37

Q. 확장형 vs 수정형 구분은?

Q. exact vs approximate 선택 기준?

알고리즘	방식	시간복잡도	정확성	특징
Backtracking	partial 확장 (DFS)	$O(\|D\|^n)$	✓ Exact	MCV·LCV·FC·AC-3로 가지치기
Beam search	partial 확장 (BFS)	$O(n\|D\|K \log K\|D\|)$	✗ Approximate	K=1 greedy, K=∞ BFS
ICM	complete 수정	$O(n\cdot\|D\|\cdot\text{iter})$	✗ Approximate	수렴보장, local opt 가능
Gibbs sampling	complete 수정 (확률)	$O(n\cdot\|D\|\cdot\text{iter})$	✗ Approximate	ICM 확률적 버전

Factor graph 최대 가중치 배정 알고리즘 비교

★ **필수암기** (1) **Partial 확장**: Backtracking(exact·지수) · Beam(approx·선형). (2) **Complete 수정**: ICM(approx·결정론) · Gibbs(approx·확률론). AC-3 후에도 global inconsistency 미감지 가능 — arc≠global.

요약 Backtracking(MCV·LCV·FC·AC-3 가지치기, exact·지수) / Beam(approximate·선형·K) = **partial assignment 확장** · ICM = **complete assignment 반복 수정**(수렴 O·전역최적 X) · AC-3는 arc-consistent여도 global inconsistency 미감지.

Ch 13 Markov Networks — 마르코프 네트워크

factor graph + 확률 정규화 · Gibbs 샘플링 · 조건부 독립 · Markov blanket

Markov Network 정의 p.7-9

Q. factor graph와의 차이 (정규화 Z)?

Q. P(x)=Weight(x)/Z의 의미는?

Q. partition function Z란?

Markov Network (마르코프 네트워크) — factor graph에 **확률 정규화**를 더한 것. 랜덤 변수 $X=(X_1,\dots,X_n)$ 위의 결합 분포를 정의.

Weight → 확률 ÷ Z로 정규화

- 정의: P(X=x) = Weight(x) / Z — Z는 partition function (정규화 상수)
- Z = Σ_x Weight(x), $\texttt{Weight}(x) = \prod_i f_i(x_i)$
- Weight는 *음수 불가*(각 factor ≥ 0); Z > 0이면 유효 분포
- CSP와 달리 최대 가중치 배정 대신 주변 확률 $P(X_i=v)$를 계산

$$\mathbb{P}(X = x) = \frac{\text{Weight}(x)}{Z}$$

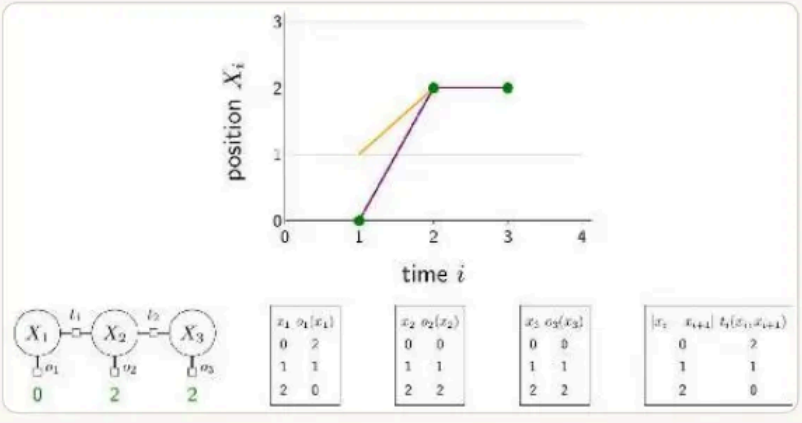
x_1	x_2	x_3	Weight(x)	$\mathbb{P}(X=x)$
0	1	1	4	0.15
0	1	2	4	0.15
1	1	1	4	0.15
1	1	2	4	0.15
1	2	1	2	0.08
1	2	2	8	0.31

$$Z = 4 + 4 + 4 + 4 + 2 + 8 = 26$$

Markov network 정의 수식 + 확률 분포표 (p.8)

Object Tracking 예제 — 위치 $X_1,X_2,X_3 \in \{0,1,2\}$, 관측 $o_1=0, o_2=2, o_3=2$.

- observation factor: $o_i(x_i)$ — 관측값과 일치할수록 높은 가중치
- transition factor: $t_i(x_i, x_{i+1}) = |x_i - x_{i+1}| + 1$ 의 역수에 비례 — 인접 위치 선호
- 최대 가중치 배정 {X_1:1,X_2:2,X_3:2}, Z=26
- $P(X_1=1) = 0.62 \neq$ argmax → 주변 확률 ≠ max 배정



Object tracking factor graph + 팩터 테이블 (p.6)

주변 확률 (Marginal Probability) — 변수 $X_i=v$의 주변 확률: $P(X_i=v) = \Sigma_{x:x_i=v} P(X=x)$

- 전체 결합 분포를 X_i 이외 변수에 대해 *합산(marginalize)*
- 직접 계산은 지수적 — 효율적 방법: Gibbs sampling

Gibbs Sampling p.10-12 · 주변확률 근사 알고리즘

Q. Gibbs sampling 한 스텝은?

Q. 조건부 분포로 무엇을 추정?

Q. 수렴(stationary)의 의미는?

Gibbs Sampling 아이디어 — 완전 배정을 무작위 초기화 후, 변수를 하나씩 **조건부 분포로 재샘플링**. 충분히 반복하면 카운트가 *실제 주변 확률로 수렴*.

- 조건부: $P(X_i=v \mid X_{-i}=x_{-i}) \propto$ `Σ_f Weight(f factors containing Xᵢ)`
- 수렴 충분조건: 모든 가중치 > 0 (기술적 조건)
- 수렴 속도: **지수 시간이 걸릴 수 있음** (보장 없음)

Gibbs Sampling

```
Initialize X ← random complete assignment
for t = 1 to T do
  for each variable Xᵢ do
    sample Xᵢ ~ P(Xᵢ | X_{-i} = x_{-i})    ← 나머지 고정
    count[i][xᵢ] += 1
Estimate P(Xᵢ=v) ≈ count[i][v] / Σ_v' count[i][v']
```

한 변수 샘플링: 연결된 factor들의 가중치 합산 → 정규화. Object tracking 예: $P(X_2=0) \propto 2$, $P(X_2=1) \propto 4$, $P(X_2=2) \propto 4$ → prob 0.2/0.4/0.4

ICM vs Gibbs — Search vs Sampling — 같은 factor graph에서 목적이 다른 두 알고리즘.

	ICM (Iterated Conditional Modes)	Gibbs Sampling
목적	최대 가중치 배정 (MAP)	주변 확률 $P(X_i=v)$
각 step	최고 가중치 값 선택	조건부 분포로 샘플링
수렴	지역 최적으로 수렴	올바른 주변확률로 수렴*
CSP 대응	Maximum weight assignment	Marginal probabilities

*모든 가중치 > 0 등 기술 조건 하. 둘 다 지수 시간 가능.

⚠ **함정** ICM은 local optimum에 빠진다 — 초기 배정에 따라 결과가 다름. Gibbs는 수렴하면 *전역적으로 올바른* 주변 확률을 복원하지만, 수렴이 **지수적으로 느릴 수 있다**.

Image Denoising 응용 — 픽셀 $X_i \in \{0,1\}$, 관측 $o_i(x_i)=[x_i=\text{관측값}]$, 인접 픽셀 factor $t_{ij}(x_i,x_j)=[x_i=x_j]+1$.

- Gibbs: 각 픽셀을 *나머지 픽셀 고정* 하에 조건부 샘플링
- 예: v=0 → weight=2, v=1 → weight=8 → $P(X_i=1 \mid \text{rest})=0.8$

독립성 & 조건부 독립 p.15-23 · 그래프 구조 활용

Q. 조건부 독립 A⊥B|C 정의는?

Q. 경로 차단(separation) 판정은?

Independence (독립) — 변수 집합 A,B가 **연결 간선이 없으면** $A \perp B$.

- 그래프적: A와 B 사이에 *edge 없음* → 별도로 최적화 가능
- 예: 호주 지도 채색에서 Tasmania {T}는 나머지와 독립
- 독립이면 **분리해서 풀면 지수적 절감**

Conditioning (조건화) — 변수 $X_i=v$를 **고정** → X_i와 연결된 binary factor들을 unary factor로 교체 → 그래프 disconnected 유도.

변수 제거 → 그래프 분리

- 절차: $X_i=v$ 고정 → X_i에 연결된 모든 factor $f_1,\cdots,f_k$ 제거
- 새 factor 추가: `g_j(x_{-i}) = f_j(x_{-i}, v)` for $j=1\cdots k$
- 그래프적으로: X_i 노드와 연결 간선을 모두 *제거*